Netflix Prize Data Mining Project

Dataset Overview

- **Netflix Prize Dataset**: 100,480,507+ ratings from 480,189 users across 17,770 movies
- Rating scale: 1-5 stars, with dates (1998-2005)
- Files: `training_set/` (17,770 files, one per movie), `movie_titles.txt`, `probe.txt` (testing), `qualifying.txt`
- Format: each movie file starts with `MovieID:`, then lines of `CustomerID,Rating,Date`

Business Problem

"How can a streaming platform leverage user rating behavior to
(1) predict what users will rate unwatched movies,
(2) discover meaningful user segments and movie clusters to inform content strategy, and (3) segment users by engagement level using RFM analysis to optimize retention?"

This framing ties all 3 data mining tasks to real business outcomes – recommendation accuracy, content investment strategy, and customer lifecycle management.

Phase 1: Data Loading and Preprocessing

Since the dataset has 17,770 separate files totaling ~2GB, efficient data loading is critical.

- Parse all `mv_*.txt` files into a single Pandas DataFrame with columns: `[MovieID, CustomerID, Rating, Date]`
- Merge with `movie_titles.txt` to get `[YearOfRelease, Title]`
- Convert `Date` to datetime, extract `Year`, `Month`, `DayOfWeek`
- Save consolidated data as a Parquet file for fast reloading
- Parse `probe.txt` to create a probe DataFrame for model evaluation
- Sampling strategy: for development, use a random 10% sample; for final results, use full data

---

Phase 2: Exploratory Data Analysis (EDA)

Deep EDA is a major scoring criterion. Cover these dimensions:

2.1 Rating Distribution Analysis

- Overall rating distribution (histogram) – expect skew toward higher ratings
- Rating distribution per year (how has rating behavior changed over time?)
- Average rating by movie release year (do older films get rated differently?)

2.2 User Behavior Analysis

- Distribution of number of ratings per user (log-scale histogram)
- Distribution of average rating per user
- Power-law analysis: show that a small fraction of users contribute most ratings
- User activity timeline: when do users rate most? (day of week, month, year trends)

2.3 Movie Popularity Analysis

- Distribution of number of ratings per movie
- Top 20 most-rated movies vs. top 20 highest-rated movies (with minimum threshold)
- Long-tail analysis: most movies have very few ratings
- Rating count vs. average rating scatter plot

2.4 Temporal Patterns

- Rating volume over time (monthly time series)

- Average rating drift over time (temporal bias)
- Heatmap of rating volume by month and year

2.5 Correlation and Statistical Tests

- Correlation between movie age and average rating
- Correlation between number of ratings and average rating
- Statistical summary tables

Key libraries: `matplotlib`, `seaborn`, `plotly` for interactive visualizations

---

Phase 3: Data Mining Problem 1 – Recommendation Engine (Collaborative Filtering)

This is the **highest-impact task** for this dataset and directly addresses the Netflix Prize challenge.

3.1 Baseline Model

- **Global Mean**: predict the global average rating for all entries
- **User + Movie Bias Model**: `prediction = global_mean + user_bias[u] + movie_bias[m]`
- Evaluate RMSE on probe set; the bias model alone should achieve ~0.98 RMSE

3.2 Matrix Factorization (SVD)

- Use the `surprise` library (`SVD`, `SVDpp`, `NMF`)
- Alternatively, implement basic SVD using `scipy.sparse.linalg.svds` on the user-item matrix
- Hyperparameter tuning: latent factors (20, 50, 100), learning rate, regularization
- Cross-validation with 5-fold or train/probe split

3.3 Neighborhood-Based Methods (for comparison)

- User-based collaborative filtering (cosine similarity, top-K neighbors)
- Item-based collaborative filtering
- Use `surprise.KNNBaseline` for efficient implementation

3.4 Evaluation

- RMSE on probe set (primary metric)
- MAE as secondary metric
- Comparison table of all models
- Netflix's Cinematch baseline RMSE was ~0.9514 on probe; aim to beat it

Key libraries: `surprise`, `scipy`, `sklearn`

Phase 4: Data Mining Problem 2 – Clustering (User and Movie Segmentation)

4.1 Feature Engineering

- **User features**: number of ratings, average rating, rating std, date range, most common rating, % of 5-star ratings
- **Movie features**: number of ratings, average rating, rating std, year of release, rating skewness

4.2 User Clustering

- Standardize features with `StandardScaler`
- Apply PCA to reduce dimensionality to 2-3 components for visualization
- K-Means clustering (use Elbow method and Silhouette score to pick K, likely 4-6)

- Interpret clusters: “Harsh Critics”, “Generous Raters”, “Casual Users”, “Power Users”, etc.
- Visualize with scatter plot colored by cluster

4.3 Movie Clustering

- Build movie feature vectors (rating statistics + year)
- Optionally use rating profile vectors (distribution of 1-5 stars)
- K-Means or hierarchical clustering
- Interpret: “Blockbusters”, “Niche Favorites”, “Polarizing Films”, “Forgotten Films”
- Dendrogram for hierarchical clustering

4.4 Advanced (bonus creativity points)

- Try DBSCAN to find outlier users/movies
- t-SNE visualization for high-dimensional rating profiles
- Compare K-Means vs. hierarchical vs. DBSCAN results

Key libraries: `sklearn.cluster`, `sklearn.decomposition`, `sklearn.preprocessing`

Phase 5: Data Mining Problem 3 – RFM Analysis (Customer Segmentation)

RFM analysis is a classic business analytics technique that maps perfectly to this dataset.

5.1 RFM Metric Definition

- **Recency (R)**: Days since user’s most recent rating (relative to dataset end date 2005-12-31)
- **Frequency (F)**: Total number of ratings by the user
- **Monetary (M)**: Average rating given by the user (proxy for engagement quality)

5.2 RFM Scoring

- Divide each metric into quintiles (1-5 score)
- Create composite RFM score per user
- Segment users into groups: “Champions” (high R, F, M), “At Risk” (low R, high F), “Lost” (low R, low F), etc.

5.3 RFM Visualization

- 3D scatter plot of R, F, M colored by segment
- Heatmap of segment counts
- Box plots comparing segments across R, F, M dimensions
- Segment size distribution (treemap or bar chart)

5.4 Business Insights

- Actionable recommendations per segment (e.g., re-engagement campaigns for “At Risk” users)
- Cross-reference RFM segments with clustering results from Phase 4

Key libraries: `pandas`, `matplotlib`, `seaborn`, `plotly`

Phase 6: Project Deliverables

6.1 Code Structure

```
final_project/
  notebooks/
    01_data_loading.ipynb         # Load, parse, consolidate data
    02_eda.ipynb                  # Full EDA with visualizations
    03_recommendation_engine.ipynb # CF models and evaluation
    04_clustering.ipynb           # User/movie clustering
```

```
    05_rfm_analysis.ipynb         # RFM segmentation
  data/
    ratings.parquet               # Consolidated dataset
  requirements.txt                # All dependencies
  README.md                       # Project overview
```

6.2 Presentation Slides (recommended structure)

1. Title + Team
2. Business Problem + Dataset Overview
3. EDA Highlights (4-5 slides with best visualizations)
4. Recommendation Engine: approach, models, RMSE comparison
5. Clustering: methodology, visualization, interpretation
6. RFM Analysis: segmentation, business insights
7. Conclusions + Future Work

6.3 Dependencies

- `pandas`, `numpy`, `matplotlib`, `seaborn`, `plotly`
- `scikit-learn` (clustering, PCA, metrics)
- `scikit-surprise` (collaborative filtering)
- `scipy` (sparse matrices, statistics)
- `pyarrow` (Parquet I/O)
- `tqdm` (progress bars for data loading)

Scoring Strategy Summary

Criterion	How We Address It
Dataset complexity	Netflix Prize – 100M+ ratings, well-known benchmark
Effectiveness of techniques	3 techniques with proper evaluation metrics
Depth of EDA	5 dimensions of analysis with rich visualizations
Presentation clarity	Modular notebooks, clear slide structure
Code quality	Annotated notebooks, requirements.txt, README
Creativity	RFM on a ratings dataset, cross-referencing segments with clusters, temporal analysis